

RFC L10-001

Selective Disclosure Layer

Technical Specification · v1.0 · FROZEN

nfra > L2 Merkle > L3 Exec > L4 tECDSA > L5 Scale > L6 SSI > L7 AI Gov > L8 OTS > L9 Hub > L10

Document ID	RFC X39MATRIX-L10-001
Version	1.0 (FROZEN)
Architect	Jose Luis Olivares Esteban
Contact	grants@x39matrix.org
PGP fingerprint	C3E062EB251A11851C0B4FFD06870F0655D5BBE8
Sealed (UTC)	2026-06-24
License (document)	CC-BY-SA 4.0
License (code)	MIT OR Apache-2.0
Derived from (YAML)	3c8ca22017a92d3de2dee014e5820cf6...

BUILD DETERMINISTA · reportlab invariant=1 · reproducible bit a bit

Anchored to Bitcoin via OpenTimestamps · Signed locally with PGP · Published at <https://x39matrix.org>

Indice

§1	Resumen ejecutivo	3
§2	Notacion y preliminares matematicos	4
§3	Parametros Winterfell (locked)	5
§4	Wire format (JSON-LD / CBOR) + firma PQ	6
§5	Circuitos C1-C5 (pseudocodigo)	7-8
§6	Modulo G · Threat Model formal	9-10
§7	Integracion con L1-L9 + canister API	11
§8	Test vectors (KAT) anclados	12
§9	Roadmap M4 · 12 sprints	13
§10	Reproducibilidad y PUBLIC_VERIFY	14
§11	Referencias normativas	15
§12	Appendice A: derivacion del hash YAML	15

§1 Resumen ejecutivo

Este RFC especifica formalmente la Capa 10 (Selective Disclosure) del protocolo X-39MATRIX. Su mision es cerrar el unico hueco de privacidad residual de las nueve capas previas: permitir demostrar la veracidad de afirmaciones sobre datos privados sin revelar dichos datos. Logra esto mediante pruebas Zero-Knowledge transparentes y post-cuanticas (zk-STARK) implementadas sobre Winterfell v0.10.x, complementadas con receipts de risc0 zkVM para casos de prueba de ejecucion (L7 AI Governance).

1.1 Doctrina

Puebas transparentes: sin trusted setup. Toda parametrizacion es publica.

Hash-only assumptions: SHAKE-256, Rescue-Prime. Sin lattices, sin discrete log.

Reproducibilidad bit a bit: cada artefacto (YAML, RFC, codigo) debe regenerarse identico.

Anclaje en Bitcoin: cada commitment relevante se sella via OpenTimestamps en bloque BTC.

Soberania del operador: el secret nunca abandona el cliente; el canister es stateless.

1.2 Objetivos de seguridad cubiertos

Selective Disclosure : probar atributo X sin entregar el documento completo.

Compliance privado : demostrar cumplimiento normativo sin doxxing del operador.

Anti-Sybil sin KYC : unicidad humana via nullifier; sin almacenamiento de identidad.

Pruueba de inferencia : verificar matematicamente la ejecucion del canister AI (L7).

Range proofs BTC : probar pago de al menos N satoshis sin revelar la direccion.

1.3 No-objetivos (explicitamente fuera del alcance v1.0)

Privacidad de red (Tor, mixnets, Nym): no es responsabilidad de la Capa 10.

Ocultacion del gasto BTC end-to-end: cubre solo range proofs, no privacy coins.

Recursion / proof aggregation: planificado para M5 con Plonky3.

SNARKs (Noir): reservado a v1.1, solo como opt-in para casos de tamano-criticos.

§2 Notacion y preliminares matematicos

2.1 Notacion

λ	parametro de seguridad (= 128 bits)
F	campo finito Goldilocks ($p = 2^{64} - 2^{32} + 1$)
H	funcion hash STARK-friendly (Rescue-Prime Rp64_256)
HSEC	funcion hash externa (SHAKE-256, FIPS-202)
P	Probador (Prover)
V	Verificador (canister x39_zk_verifier en ICP)
C	circuito aritmetico (AIR) sobre F
π	prueba STARK emitida por P
x	witness privado (input secreto del Prover)
y	output publico (resultado verificable)
pk_{gov}	clave publica de una autoridad emisora (firma de credenciales)
σ	firma SLH-DSA-SHAKE-256s (FIPS-205) sobre el wrapper del proof_message
ϵ	probabilidad despreciable, $\epsilon < 2^{-\lambda}$

2.2 Definicion formal (Goldwasser-Micali-Rackoff, 1985)

Un sistema de prueba interactivo (P, V) para una relacion R es una prueba de conocimiento cero si satisface tres propiedades:

COMPLETENESS	: forall (x, y) in R . $\Pr[V(y, \pi) = 1] \geq 1 - \epsilon$
SOUNDNESS	: forall (x, y) not in R . forall P^* . $\Pr[V(y, P^*(y)) = 1] \leq \epsilon$
ZERO-KNOWLEDGE	: exists S simulator . forall V^* . $\{ \text{trans}(P, V^*)(x, y) \} \approx \{ S(y) \}$ (indistinguibilidad computacional sobre la transcripcion completa)

2.3 Por que zk-STARK y no zk-SNARK

STARK es transparente: no requiere ceremonia de trusted setup. SNARK Groth16/PLONK si.

STARK depende solo de hashes: post-cuantica por construccion. SNARK depende de pairings.

STARK tiene complejidad poli-logaritmica en verificacion: $O(\log^2 n)$.

STARK escala mejor a circuitos grandes (millones de constraints).

Coste: pruebas STARK mas grandes (~60-250 KB vs ~200 bytes SNARK). Aceptable.

§3 Parametros Winterfell (locked)

Los parametros listados a continuacion son normativos y forman parte del freeze v1.0. Cualquier desviacion exige bump de version del RFC y regeneracion de todos los artefactos derivados.

3.1 Configuracion criptografica

```
backend      : Winterfell v0.10.x  (Rust nativo)
repo         : https://github.com/facebook/winterfell
field        : BaseElement (Goldilocks, p = 2^64 - 2^32 + 1)
hash (in-circuit) : Rescue-Prime  (Rp64_256)
hash (Merkle)   : Rescue-Prime  (Rp64_256)
hash (external) : SHAKE-256      (FIPS-202)
fri folding factor : 8
fri max remainder : 256
num queries    : 27
blowup factor   : 8
grinding bits   : 16
field extension  : QuadExtension
security target : 128-bit classical / PQ via hash-only
```

3.2 Firma PQ del wrapper proof_message

```
algorithm      : SLH-DSA-SHAKE-256s  (FIPS-205, hash-only)
signature size  : ~29 792 bytes
public key size : 64 bytes
secret key size : 128 bytes
hash function   : SHAKE-256
rationale       : maxima conservacion PQ; una sola hipotesis
                  criptografica (no se rompe SHAKE/SHA).
```

3.3 Justificacion de num_queries = 27

Con blowup = 8 (-> domain rate rho = 1/8) y grinding de 16 bits, la formula de soundness post-cuantica de FRI exige $\text{num_queries} \geq \lceil \lambda / \log_2(1/\rho) \rceil - \text{grinding_bits} / \log_2(1/\rho)$. Sustituyendo: $\lceil 128 / 3 \rceil - 16/3 \approx 43 - 6 = 37$. Con conjectural soundness de FRI (Ben-Sasson et al., 2023) se admite 27 consultas para 128-bit clasico. Para PQ estricto se recomienda subir a 37 en una variante 'paranoid mode' de la spec; aqui se elige 27 como default por equilibrio rendimiento-soundness.

§4 Wire format (JSON-LD / CBOR) + firma PQ

4.1 Estructura del proof_message (JSON-LD canonico)

```
{
  "@context": "https://x39matrix.org/contexts/l10/v1.jsonld",
  "proof_type": "zk-STARK",           // o "risc0"
  "circuit_id": "age_ge_18_v1",
  "circuit_air_sha256": "<32 bytes hex>",
  "public_inputs": [ ... ],           // arr de field elements en hex
  "public_inputs_hash": "<sha256 of canonical CBOR>",
  "proof_blob": "<base64 STARK bytes>",
  "ots_commitment": "<sha256 hex>",
  "anchor_block": null,               // se rellena tras attestation BTC
  "issuer_canister": "arn4r-lqaaa-...",
  "timestamp_utc": "2026-06-24T15:00:00Z",
  "verifier_audience": "<verifier id, opt>",
  "nullifier": "<rescue-prime output hex, opt>",
  "wrapper_signature": {
    "alg": "SLH-DSA-SHAKE-256s",
    "sig": "<base64>",
    "pubkey": "<base64>"
  }
}
```

4.2 Canonicalizacion para hash

JSON-LD se canonicaliza a CBOR usando RFC 8949 (deterministic encoding).

El hash del CBOR canonico es la entrada para ots stamp y para la firma PQ.

El hash se calcula con SHAKE-256 con output truncado a 256 bits (32 bytes).

Esto garantiza que el mismo proof produzca el mismo commitment, sin ambigüedad.

4.3 Verificacion en canister (orden estricto)

1. canonicalize(proof_message) -> CBOR bytes
2. SHAKE-256(CBOR) -> hash H
3. verify SLH-DSA-SHAKE-256s wrapper_signature on H
4. verify STARK pi against public_inputs and circuit_air_sha256
5. if nullifier present: check (nullifier, verifier_audience, epoch) is fresh
6. accept or reject

§5 Circuitos C1-C5 (pseudocodigo)

Cada circuito se especifica como un AIR (Algebraic Intermediate Representation) sobre el campo Goldilocks. Aquí se da el pseudocodigo de alto nivel; los AIRs precisos se publican como crates Rust en el repo del protocolo, firmados PGP y anclados via OTS.

C1 · `age_ge_18` · prueba edad ≥ 18 sin revelar fecha de nacimiento

```
public_inputs : btc_block_height: u64, btc_block_hash: [u8; 32]
private_inputs: date_of_birth: Field, gov_sig: Signature, gov_pubkey: Field

fn main(dob: Field, sig, pk_gov, btc_height, btc_hash) {
  assert(verify_eddsa(pk_gov, dob, sig));
  let now = derive_year_from_btc_block(btc_height, btc_hash); // ~10 min/bloque
  assert(now - dob  $\geq$  18 * SECONDS_PER_YEAR);
}
```

C2 · `jurisdiction_residence` · prueba residencia sin entregar pasaporte

```
public_inputs : target_jurisdiction_id: Field
private_inputs: passport_hash: Field, jurisdiction: Field, gov_sig: Signature

fn main(passport_hash, jur, sig, target) {
  assert(verify_eddsa(pk_gov, hash(passport_hash || jur), sig));
  assert(jur == target);
}
```

C3 · `btc_payment_range_proof` · prueba pago BTC $\geq N$ sats sin revelar address

```
public_inputs : threshold_sats: u64, btc_block_hash: [u8; 32]
private_inputs: utxo_value: u64, utxo_commitment: Field, merkle_path: Vec<Field>

fn main(value, commitment, path, threshold, btc_hash) {
  assert(verify_merkle(btc_hash, commitment, path));
  assert(value  $\geq$  threshold);
}
```

C4 · unique_human_nullifier · anti-Sybil sin KYC

```
public_inputs : verifier_audience: Field, epoch: Field, nullifier_out: Field
private_inputs: secret: Field, circuit_id: Field
```

```
fn main(secret, circuit_id, audience, epoch) -> pub Field {
  let nf = rescue_prime([secret, circuit_id, audience, epoch]);
  nf // publicado, secret nunca sale del cliente
}
```

REGLA DE EPOCA: per-verifier epoch ->

el mismo humano ante 2 verificadores distintos produce nullifiers no correlacionables.
Un verificador concreto detecta reusos del mismo humano dentro de su epoca.

C5 · ai_inference_receipt · prueba de ejecucion de modelo AI (risc0)

```
proof_system : risc0 zkVM
public_inputs : model_hash: [u8; 32], output_commitment: [u8; 32]
private_inputs : input_vector: Vec<f32>, model_weights: Vec<f32>
```

// risc0 guest program (Rust):

```
fn main() {
  let input: Vec<f32> = env::read();
  let weights_hash = env::read::<[u8; 32]>();
  assert_eq!(weights_hash, MODEL_HASH);
  let output = model_forward(&WEIGHTS, &input);
  let commitment = sha256(&output);
  env::commit(&commitment);
}
```

NOTA OPERATIVA: prover time del orden de 60-600 s para modelos pequenos.

Para modelos > 100M parametros se recomienda server-assisted proving (opt-in).

5.6 Reglas comunes a todos los circuitos

Cada circuito publica su AIR como un crate Rust firmado PGP y sellado OTS.

Cada circuito tiene su KAT (Known Answer Test) publicado para reproducibilidad.

Cualquier cambio a un circuito incrementa su sufijo (_v2, _v3) y lock nuevo del SHA.

§6 Modulo G · Threat Model formal

Esta seccion enumera los adversarios considerados por la Capa 10, sus capacidades, sus objetivos, y las mitigaciones especificadas. Un auditor externo que apruebe el RFC debe firmar tambien este modulo de amenazas.

6.1 Adversario clasico (A_C)

Recursos : computo clasico polinomial, almacenamiento polinomial.

Objetivos: (a) forjar pruebas STARK falsas, (b) extraer secret del cliente,
(c) correlacionar nullifiers entre verificadores.

Mitigacion:

- Soundness 128-bit clasica via `num_queries=27 + grinding=16`.
- secret never leaves client; canister stateless.
- per-verifier epoch -> nullifiers no correlacionables (definicion 4.4).

6.2 Adversario cuantico (A_Q)

Recursos : computo cuantico polinomial (Shor, Grover, BHT collision).

Objetivos: (a) romper la firma del wrapper `proof_message`,
(b) romper la dureza del hash subyacente del STARK,
(c) romper ECDSA en BTC (fuera de scope L10).

Mitigacion:

- Firma SLH-DSA-SHAKE-256s (FIPS-205): hash-only, sin lattice.
- Hash STARK: Rescue-Prime + Grover -> 2^{64} -> mitigado con `grinding=16` y `num_queries=27` (2^{128} trabajo cuantico estimado).
- BTC ECDSA: limitacion conocida; el operador acepta que en horizonte ~2030+ Bitcoin migrara a firmas PQ (BIP-XXX en discusion).

6.3 Probador malicioso (A_P)

Capacidades: P controla totalmente la generacion de la prueba.

Objetivos : convencer a V de una afirmacion falsa $f(x) \neq y$.

Mitigacion :

- Soundness FRI: $\Pr[V \text{ acepta }] \leq 2^{-128}$ (clasica) si la afirmacion es falsa.
- `public_inputs_hash` : V recomputa y compara; resistente a malleability.
- nullifier : reuso detectado por V si la epoca es la misma.

6.4 Verificador malicioso (A_V)

Capacidades: V intenta extraer informacion privada de la transcripcion.
Objetivos : recuperar x (witness privado) a partir de pi y public inputs.
Mitigacion :
- Zero-Knowledge property: existe simulador S que produce pi sin x.
- Argumento: cualquier informacion que V extrae, S la habria producido con solo public inputs -> V no aprende nada nuevo.

6.5 Side-channels en el cliente (A_S)

Capacidades: timing, power analysis, EM emanations.
Objetivos : extraer secret del cliente durante el proving.
Mitigacion :
- Recomendacion normativa: usar constant-time hash (Rescue-Prime CT impl).
- Recomendacion operativa: proving en aire-gapped device cuando sea critico.
- Server-assisted proving: opt-in; jamas se envia el secret en claro.

6.6 Coercion del operador (A_R, rubber-hose attack)

Capacidades: violencia fisica o legal para extraer secret y PGP.
Objetivos : sustituir al operador legitimo o forjar pruebas.
Mitigacion :
- Threshold-PGP signing (Sprint 10): firma requiere t-of-n custodios.
- Plausible deniability via decoy nullifiers (research).
- Geographic distribution: custodios en jurisdicciones independientes.

6.7 Censura del canister (A_X)

Capacidades: presion legal/economica sobre nodos ICP.
Objetivos : impedir verificaciones o forzar logs.
Mitigacion :
- x39_zk_verifier replicado en >=3 subnets ICP independientes.
- Fallback verifier WASM ejecutable offline por el auditor.
- Anclaje BTC: ningun actor puede borrar el commitment una vez anclado.

6.8 Resumen tabular de mitigaciones

Adversario	Vector	Mitigacion principal	Residual?
A_C	forge STARK	num_queries=27, grinding	epsilon
A_Q	quantum break sig	SLH-DSA-SHAKE-256s	None
A_P	malicious prover	FRI soundness 2^128	epsilon
A_V	extract witness	ZK property + simulator	None
A_S	side-channel client	CT impl + air-gap	Operational
A_R	rubber-hose	threshold-PGP (M5)	Legal/social
A_X	censorship	multi-subnet + BTC anchor	None

§7 Integracion con L1-L9 + canister API

7.1 Mapa de integracion

L1 Infra	Hospeda x39_zk_verifier en subnet de aplicacion; replicacion en 3 subnets.
L2 Identity Merkle	Hojas Merkle pueden ser commitments en vez de identidades en claro.
L3 Execution	Cada ejecucion canister puede emitir un STARK de su correccion (opt-in).
L4 tECDSA	Firma BTC condicionada a verificacion ZK de elegibilidad del firmante.
L5 Scalability	Recursion STARK agregable (M5 con Plonky3) -> 1000 -> 1 prueba.
L6 Identity SSI	DIDs emitidos con atributos selectivos: probar 'soy X' sin revelar X.
L7 AI Governance	risc0 receipts: probar 'el modelo M ejecuto con input I' sin filtrar pesos.
L8 Notarization	Cada commitment del proof_message se ancla via OpenTimestamps a BTC.
L9 Hub	Orquesta el pipeline P -> V -> commitment -> OTS -> BTC end-to-end.

7.2 Canister API (Candid interface)

```

service : {
  register_circuit   : (text, blob)      -> (variant { ok; err: text });
  list_circuits      : ()                 -> (vec record { id: text; sha256: text }) query;
  verify_proof       : (ProofMessage)    -> (variant { ok; err: text });
  submit_anchored    : (ProofMessage)    -> (variant { ok: text; err: text });
  get_proof_status   : (text)            -> (record { anchored: bool; btc_block: opt nat }) query;
  list_revocations   : ()                 -> (vec text) query;
  publish_revoke     : (text, text)      -> (variant { ok; err: text });
}

type ProofMessage = record {
  proof_type          : variant { stark; risc0 };
  circuit_id          : text;
  circuit_air_sha256  : text;
  public_inputs       : vec nat;
  public_inputs_hash  : text;
  proof_blob          : blob;
  ots_commitment      : text;
  issuer_canister     : principal;
  timestamp_utc       : text;
  verifier_audience  : opt text;
  nullifier           : opt text;
  wrapper_signature   : record { alg: text; sig: blob; pubkey: blob };
};

```

§8 Known Answer Tests (KAT) anclados

Cada circuito (C1-C5) publica al menos 10 KATs en formato JSON canonico. El conjunto de KATs se hashea con SHAKE-256, se sella con OpenTimestamps, y se firma con la clave PGP del autor. Esto garantiza que cualquier implementacion futura del circuito que pase los KATs es necesariamente compatible con la spec original.

8.1 Estructura de un KAT

```
{
  "kat_id"           : "C1_age_ge_18_kat_001",
  "circuit_id"       : "age_ge_18_v1",
  "circuit_air_sha256" : "<32 bytes hex>",
  "private_inputs"   : { "dob": "1980-05-12", ... }, // solo para test
  "public_inputs"    : { "btc_block_height": 850000, "btc_block_hash": "..." },
  "expected_proof_sha256" : "<32 bytes hex>",
  "expected_outcome"  : "accept",
  "negative_variant"  : false
}
```

8.2 Publicacion

URL : <https://x39matrix.org/L10/kat/C{1..5}/> (un JSON por KAT).

Bundle: https://x39matrix.org/L10/kat/bundle_v1.0.tar.zst + .ots + .asc.

Hash de bundle anclado en este RFC en §12 (appendice A).

Cualquier auditor reproduce: descarga bundle, ejecuta crate de tests, espera 51/51.

8.3 Politica de incompatibilidad

Un nuevo circuito que falle uno o mas KATs del set v1.0 NO PUEDE llamarse age_ge_18_v1. Debe publicarse como age_ge_18_v2 con su propio set de KATs. Esto garantiza que el protocolo evoluciona sin romper compatibilidad silenciosa.

§9 Roadmap M4 · 12 sprints

- S0** Lock parameters (este RFC). Apertura issues #spec, #threat, #pq-sig, #time-oracle.
- S1** Implementar C1 age_ge_18 con BTC block header oracle. KAT 1-10.
- S2** Implementar C2 (jurisdiction) y C3 (BTC range proof). Integracion L4 tECDSA.
- S3** Implementar C4 (nullifier). Integracion L6 SSI. Per-verifier epoch tests.
- S4** Implementar C5 risc0 AI receipt. Integracion L7 AI Governance.
- S5** Canister x39_zk_verifier en Rust + ic-cdk. Verifier SLH-DSA wrapper.
- S6** OTS pipeline + BTC anchoring queue para commitments STARK.
- S7** PUBLIC_VERIFY_LAYER10.sh + reproducibilidad CI (GitHub Actions / Forgejo).
- S8** Auditoria externa preparatoria (test vectors, KAT, threat model review).
- S9** Evaluacion Plonky3 para recursion / proof aggregation.
- S10** GDPR / EU compliance pack + threshold-PGP signing. M4 Phase Gate.
- S11** Testnet publica con C1-C5 vivos. Bug bounty 0.001 BTC inicial.
- S12** Mainnet rollout v10.0 + release publico anclado a BTC.

9.1 Hito final M4

X-39MATRIX v10.0 sera el primer protocolo soberano del mundo en combinar, en una sola arquitectura coherente y reproducible: consenso BFT post-cuantico (ICP), firma umbral ECDSA sobre Bitcoin nativo, anclaje temporal universal via OpenTimestamps, criptografia post-cuantica FIPS-203/204/205, y divulgacion selectiva zk-STARK transparente. Cinco propiedades, una sola doctrina: la soberania matematica del operador.

§10 Reproducibilidad y PUBLIC_VERIFY

Toda la doctrina de X-39MATRIX se apoya en una sola promesa operativa: 'Don't trust. Verify.' Para que esa promesa sea cumplible, cada artefacto debe ser reproducible bit a bit por cualquier auditor sin acceso privilegiado a la infraestructura del autor.

10.1 Build determinista del RFC (este documento)

```
Compilador   : Python 3.11+
Libreria     : reportlab 5.0.0 con canvas(invariant=1, pageCompression=0)
Script       : /tmp/l10_build/build_l10_rfc.py (publicado bajo MIT)
Dependencia  : ninguna fuente externa, ningun timestamp dinamico
Garantia     : mismo Python + mismo reportlab + mismo script -> mismo SHA-256
```

10.2 Script PUBLIC_VERIFY_LAYER10.sh (PASO 4)

```
Acciones del script:
1. curl -sL0 los 4 artefactos publicos (yaml, rfc.pdf, whitepaper.pdf, este script).
2. sha256sum contra hashes pineados en el propio script.
3. ots verify para cada uno (yaml.ots, rfc.pdf.ots, whitepaper.pdf.ots).
4. gpg --verify para cada uno (.asc).
5. salida final: 51/51 OK o bloqueo con codigo de error.
```

10.3 Cadena de dependencia entre artefactos



Si el SHA-256 del YAML cambia (root), todos los hijos quedan invalidados.
Esto convierte la cadena en un Merkle DAG implicito con anclaje BTC en cada nodo.

§11 Referencias normativas

[*] FIPS-202 : SHA-3 Standard (SHAKE-256). NIST, 2015.

[*] FIPS-203 : ML-KEM (Module Lattice Key Encapsulation). NIST, 2024.

[*] FIPS-204 : ML-DSA (Module Lattice Digital Signature). NIST, 2024.

[*] FIPS-205 : SLH-DSA (Stateless Hash-based Digital Signature). NIST, 2024.

[*] Ben-Sasson et al. : Scalable Zero Knowledge with no Trusted Setup. CRYPTO 2018.

[*] Ben-Sasson et al. : STIR: Reed-Solomon Proximity Testing with Fewer Queries. ePrint 2023.

[*] Goldwasser, Micali, Rackoff : The Knowledge Complexity of Interactive Proof Systems. SIAM 1989.

[*] Aggelos Kiayias et al. : Cryptocurrency Composability. Eurocrypt 2020.

[*] RFC 8949 : Concise Binary Object Representation (CBOR). IETF, 2020.

[*] BIP 340/341/342 : Schnorr / Taproot. Bitcoin Improvement Proposals, 2020-21.

[*] OpenTimestamps : <https://opentimestamps.org/>. Peter Todd, 2016+.

[*] DFINITY : Internet Computer Protocol whitepaper, 2021.

[*] Winterfell : <https://github.com/facebook/winterfell> (Polygon-Miden ecosystem).

[*] risc0 : <https://risc0.com> (Zero-Knowledge zkVM in RISC-V).

§12 Appendice A · derivacion del hash YAML

Este RFC declara dependencia normativa del documento X39MATRIX_LAYER10_DECISIONS.yaml por su SHA-256 exacto. La cadena de verificacion es:

```
YAML SHA-256 : 3c8ca22017a92d3de2dee014e5820cf6f61b82e4af4e6503e828393d489828f5
YAML URL      : https://x39matrix.org/X39MATRIX_LAYER10_DECISIONS.yaml
YAML OTS      : https://x39matrix.org/X39MATRIX_LAYER10_DECISIONS.yaml.ots
YAML PGP sig   : https://x39matrix.org/X39MATRIX_LAYER10_DECISIONS.yaml.asc
```

```
Comando de validacion (auditor):
curl -sL https://x39matrix.org/X39MATRIX_LAYER10_DECISIONS.yaml | sha256sum
-> debe imprimir el hash de arriba.
```

Cierre formal del RFC

El presente documento queda FROZEN en su version 1.0 con fecha 2026-06-24 UTC. Cualquier modificacion exige bump de version, regeneracion deterministica, nuevo sellado OTS y nueva firma PGP. La identidad criptografica del autor es C3E062EB251A11851C0B4FFD06870F0655D5BBE8.

X-39MATRIX · RFC L10-001 · v1.0

Selective Disclosure Layer · Winterfell · risc0 · Post-Quantum · Bitcoin-anchored

```
H(documento) -> SHAKE-256          -> OpenTimestamps -> Bitcoin PoW
H(circuito)   -> Rescue-Prime       -> STARK proof   -> Verifier canister
H(identidad)  -> Rescue-Prime       -> selective open -> zero-knowledge
H(wrapper)    -> SLH-DSA-SHAKE      -> signature     -> post-quantum
```

"Cypherpunks write code. Cypherpunks build math."

Eric Hughes, 1993 — honrado en cada hash de este documento.

Jose Luis Olivares Esteban

Arquitecto unico · Sevilla, 24 de junio de 2026

PGP: C3E062EB251A11851C0B4FFD06870F0655D5BBE8

grants@x39matrix.org · <https://x39matrix.org>

BUILD DETERMINISTA · reportlab invariant=1 · reproducible bit a bit

Derived from YAML SHA-256: 3c8ca22017a92d3de2dee014e5820cf6f61b82e4af4e6503...